

Práctica 1

Diseño sostenible y principios SOLID

Arquitectura de Software I

Diagnóstico Cuantitativo, Refactorización SOLID y Registros de Decisiones (ADRs)

Propósito

Analizar una solución en C# que presenta problemas de diseño y modularidad, identificar las violaciones a los principios SOLID, medir sus características de cohesión y acoplamiento, y realizar una refactorización que mejore su estructura.

Objetivo

Evaluar la capacidad del equipo de desarrollo para diagnosticar, refactorizar y documentar

cuantitativamente una solución de software orientada a objetos en C#, aplicando principios de

diseño sostenible y estándares internacionales de arquitectura:

1. **Identificar y diagnosticar** antipatrones de diseño y violaciones explícitas a los 5 principios SOLID (SRP, OCP, LSP, ISP, DIP) en un módulo legado.
2. **Cuantificar** la calidad de la arquitectura existente mediante métricas formales de cohesión (LCOM96b), acoplamiento (Ca, Ce), inestabilidad (I), abstracción (A) y distancia a la secuencia principal (D).
3. **Refactorizar** la solución en C# aplicando buenas prácticas que solucionen violaciones de principios SOLID.
4. **Justificar y documentar** las decisiones de diseño mediante registros de decisiones de arquitectura (ADRs) bajo la norma ISO/IEC/IEEE 42010:2022.

Descripción de caso legacy

La clínica odontológica usa un sistema para administrar sus citas.

El sistema permite:

- Agendar citas.
- Cancelar citas.
- Consultar citas.
- Validar disponibilidad del odontólogo.
- Calcular el valor de una cita.
- Enviar notificaciones al paciente.
- Almacenar las citas.

El equipo de arquitectura recibe el código fuente del módulo central de la clínica odontológica. La clase principal GestorCitasOdontologicas ha acumulado responsabilidades diversas a lo largo de los años, convirtiéndose en una God Class monolítica con serios problemas de mantenibilidad:

- ❖ **Agendamiento y Reglas de Negocio:** Valida disponibilidad del odontólogo y calcula el costo del abono/copago utilizando un switch gigante dependiente del tipo de especialidad (Ortodoncia, Endodoncia, Odontopediatría, Cirugía) y convenio (Particular, EPS, Prepagada).
- ❖ **Políticas de Cancelación:** Procesa cancelaciones aplicando multas dinámicas si la cita se cancela con menos de 24 horas de antelación o según el tipo de tratamiento.
- ❖ **Acceso a Datos e Infraestructura:** Realiza persistencia directa en SQL Server mediante SqlCommand embebido y envía recordatorios inmediatos por SMS (Twilio API) y Correo (SMTP) hardcodedos en la misma clase.

Estructura de Entregables del Trabajo

El proyecto debe ser estructurado y entregado en un archivo comprimido o repositorio público de GitHub con el siguiente árbol de carpetas:

```
/
├── src/
│   ├── DentalCare.Legacy/ <-- Código base inicial suministrado
│   └── DentalCare.Refactored/ <-- Solución refactorizada desacoplada
├── docs/
│   ├── adr/ <-- Carpeta con los archivos Markdown de ADRs
│   │   ├── ADR-001-inversion-dependencias-persistencia-notificaciones.md
│   │   └── ADR-002-ajuste-politicas-cancelacion-tarifas.md
│   └── Informe_Metricas.pdf
```

Fases del Trabajo y Contenido Requerido

Fase 1: Diagnóstico SOLID y Métricas Baseline

1. **Matriz de Violaciones SOLID:** Identificar y describir dónde y cómo se violan los 5 principios (SRP, OCP, LSP, ISP, DIP) en el código legado, citando las líneas exactas de código.
2. **Línea Base Cuantitativa (Baseline):** Calcular sobre la clase monolítica GestorCitasOdontologicas:

- a. **LCOM96b:** Mostrar el desarrollo matemático detallado aplicando la fórmula:

$$LCOM96b = \frac{1}{a} * \sum_{j=1}^a \left[\frac{m - \mu(A_j)}{m} \right]$$

- b. **Acoplamiento Ca y Ce:** Conexiones entrantes y salientes (detallar cuales son).
- c. **Inestabilidad (I), Abstracción (A) y Distancia (D):** Determinar la posición en el plano A vs. I y verificar si cae en la Zona de Dolor (Zone of Pain: A=0, I=0).

$$I = \frac{C_e}{C_e + C_a},$$

$$A = \frac{\sum m_a}{\sum m_c}, \quad D = |A + I| - 1$$

Fase 2: Refactorización y Diseño Sostenible

Refactorizar el proyecto en la carpeta DentaCare.Refactored aplicando los principios SOLID

Fase 3: Recálculo de Métricas e Impacto Cuantitativo

Construir una Tabla Comparativa Antes vs. Después en el informe respaldando las mejoras teniendo en cuenta las virtudes adquiridas por SOLID y la métricas de modularidad (cohesión y acoplamiento).

Fase 4: Documentación de Decisiones Arquitectónicas (ADRs)

Redactar mínimo los ADRs en formato Markdown alineados con el estándar ISO/IEC/IEEE 42010:2022 de las decisiones tomadas en la refactorización.

Rúbrica de Evaluación

Criterio	Peso	Excelente (5.0)	Aceptable (3.5)	Insuficiente (1.0 - 2.0)
Diagnóstico SOLID	25%	Identifica las 5 violaciones SOLID.	Identifica parcialmente las violaciones SOLID.	Diagnóstico ambiguo, erróneo o incompleto.
Cálculo de Métricas Baseline vs Refactorizado	25%	Muestra el desarrollo matemático de LCOM96b, I, A, D y tabla comparativa impecable.	Calcula las métricas pero omite fórmulas o recálculo posterior.	No presenta cálculos ni tabla comparativa cuantitativa.
Calidad del Código Refactorizado C#	30%	Código C# limpio, desacoplado con DI, interfaces, etc.	Refactoriza el código pero mantiene dependencias concretas.	El código no compila o mantiene la clase monolítica.
Rigor en ADRs (ISO 42010)	20%	ADRs redactados con notación estándar, análisis profundo de trade-offs y gobernanza.	Presenta ADRs pero carecen de consecuencias (trade-offs) o estatus.	No entrega ADRs o los redacta como simple resumen.

Anexo: Código C# Legado Suministrado (DentalCare.Legacy)

Se comparte el proyecto en zip "DentalCare.zip"